

A lot of 0's received when using cudaMemcpy()

Asked 8 years, 3 months ago Modified 8 years, 3 months ago Viewed 1k times

I've just started to learn CUDA and i wanted to fill an array (a 2D array represented as a 1D array) with random numbers. I followed another posts in order to generate random numbers, but i don't know if there is a problem with the generation of numbers or with the memory recovering from the device or anything else. The problem is that, though i have tried to fill any cell of the array with the id of the thread that is attending it in order to see the results after copying into the host memory, i receive an array that is filled with 0 in any position after recovering the data with `cudaMemcpy()`.

I'm programming on Visual Studio 2013, with cuda 7.5, on a i5 2500k as my processor and a 960 GTX graphic card.

Here is the main and the method where i try to fill it. I'll update the cuRand Initialization too. If you need to see something else, just tell me.

```

...global __void setup_cudaRand(cuRandState * state, unsigned long seed)
{
    int id = threadIdx.x;
    curand_init(seed, id, 0, &state[id]);
}

...global __void polarInt __adn, curandState * state){
    curandState localState = state[threadIdx.x];
    int random = cuRandn(&localState);
    adn[threadIdx.x] = random;
    // It doesn't mind if I use the following instruction, the result is a lot of
0's
    __adn[threadIdx.x] = threadIdx.x;
}

int main()
{
    const int adnLength = NUMCUDASOMAS + SIZECUDASOMAS; // 256 * 128 (32.768)
    const size_t adnSize = adnLength * sizeof(int);
    int *adnCPU=(int*)malloc(adnSize);
    int *adnDevice;
    cudaError_t error = cudaSetDevice(0);
    if (error != cudaSuccess)
        exit(EXIT_FAILURE);

    curandState * randState;
    error = cudaMalloc(&randState, adnLength * sizeof(curandState));
    if (error != cudaSuccess){
        cudaFree(randState);
        exit(EXIT_FAILURE);
    }

    //Here is initialized cudaRand
    setup_cudaRand __adn, adnLength == > > (randState, unsigned(time(NULL)));

    error = cudaMalloc(&adnDevice, adnSize);
    if (error == cudaErrorFreeMemoryAllocation) // cudaSuccess{
        cudaFree(adnDevice);
        cudaFree(randState);
        printf("\n error:");
        exit(EXIT_FAILURE);
    }

    polar __adn, adnLength == > > (adnDevice, randState);
    error = cudaMemcpy(adnDevice, adnDevice, adnSize, cudaMemcpyDeviceToHost);
    //After here, for any i, adnCPU[i] is 0 and I cannot figure what is wrong
    if (error != cudaSuccess){
        for (int i = 0; i < NUMCUDASOMAS; i++){
            for (int j = 0; j < SIZECUDASOMAS; j++){
                printf("%d", adnCPU[i+(j*SIZECUDASOMAS)]);
            }
            printf("\n");
        }
    }

    return 0;
}

```

EDIT after answer solved: There was a particularity over the answer given, and is that you need a lower number of threads (half of that quantity worked for me) in order to seed correctly the random numbers with cuRand. For some reason, i could create the threads perfectly but i couldn't seed the pseudo-random algorithm generator.

arrays parallel-processing cuda gpu [Edit tags](#)

Share Edit Follow Close Flag edited May 8, 2016 at 12:23 asked May 7, 2016 at 8:27

The code in this question is incomplete and can't compiled or run. The (now modified) question you are asking about can only be answered definitely by seeing the code for `setup_cuRand`. But please don't keep editing this question to the point where the accepted answer is no longer relevant. If you can a question about seeding and initialising the cuRand generator, then by all means ask it. But do it in a new question. This isn't your person help thread, it is a question and answer pair which is intended to be useful to everyone, not just you. – talonmies May 8, 2016 at 10:41

⚑ @Talonmies Sorry if I didn't follow the correct way to solve my errors. I had another little problem in the code and notified it before someone used the code. After that I found the solution and posted too. I'll change the title one last time because a previous one was the most relevant from the problem that I finally had (and that I didn't know that was that). I want to apologize everyone if I had a behaviour that could harm the community's learning. — Kaostis May 8, 2016 at 12:13

1 Answer

Sorted by: Reset to default Date modified (newest first)

The maximum number of threads per block is 1024 on your hardware, hence, you may not schedule a call with `adnLength` if it is larger than 1024.

2 The error you are having is most probably a call configuration error, and it is returned by `cudaPeekAtLastError`, as it occurs before any GP after the triple angled-bracket call. Indeed `cudaMemcpy` may not return it, even though it returns `error` from previous asynchronous calls.

The error that may occur is `cudaErrorLaunchOutOfResources`.

Share Edit Follow Flag

answered May 7, 2016 at 8:56
 Florent DUGUET
 2,866 18 29

[↩](#) In order to solve this, I've just run 256 threads and loop 128 times and it really worked. Thanks a lot. - [Kaostis](#) May 7, 2016 at 9:03
[\[in\]](#)

[↩](#) Is there any problem with the way I initialize curand? The numbers I obtain are the same for each thread. - [Kaostis](#) May 7, 2016 at 9:53
[\[in\]](#)

[↩](#) You should set up a seeding scheme such that each thread uses a different value. The notion of subsequence is of interest to you. Dig into curand documentation and maybe post another question if you are still stuck. - [Florent DUGUET](#) May 7, 2016 at 16:01
[\[in\]](#)

[↩](#) The problem was, another. For some reason, my `curand` could only operate with half of the quantity of maximum threads (1024 threads is my max val so 512 threads) because the pseudo-random generator from `curand` just used to work when more of those threads were running. - [Kaostis](#) May 8, 2016 at 9:32
[\[in\]](#)